

Firefox Crash Fix/Regression Test-Validation Benchmark

Overview

[Pipeline Stages and Dataset Statistics](#)

[Bug Categories in this Dataset](#)

[Stage 1 — Extract crash bugs](#)

[Stage 2 — Extract stack traces](#)

[Stage 3 — Filter by regression \(noise-reduction signal only, not used for test validation\)](#)

[Stage 4 — Find fixing commit and its parent commit](#)

[Test Execution Methodology: Patching the Fixing Commit's Test onto the Parent](#)

[Stage 5 — Check for test files on the fixing commit](#)

[Stage 6 — Build readiness filter](#)

[Stage 7 — Artifact build attempt](#)

[Stage 8 — Full build, split by Python era](#)

[Current status of the Python 3 batch](#)

[What We're Asking Mozilla Developers](#)

[1. The Build Blocker: cbindgen TOML Parse Failures](#)

[2. Is There a Way to Avoid Manually Discovering Dependencies Per Test Framework?](#)

[We're following Mozilla's own published documentation for running each test suite – especially:](#)

[Pipeline Summary Diagram](#)

Overview

We are building a benchmark that empirically validates Core and Firefox crash fixes, using a structure similar to established software-defect benchmarks in the research community (e.g., Defects4j for Java): each crash bug's fixing commit is paired with its **immediate parent commit** (the pre-fix, "buggy" revision), and we prove causality empirically by running the bug's own test file(s) against both revisions — expecting the test to **pass on the fix** and **fail on the parent**. Unlike Defects4J's single-language, manually-minimized bug set, our benchmark is built from Core and Firefox's real, unminimized commit history across a multi-language (C++, Rust, JavaScript) production codebase spanning nearly a decade of toolchain changes.

We've applied this pipeline to a dataset of 717 Firefox/Core crash bugs mined from Bugzilla and are currently mid-way through building and testing 183 bugs from the "Python 3 era" of the codebase (2022-2026). We're documenting our full methodology below, along with two open problems we'd value Mozilla engineers' input on: a recurring build failure affecting a large share of our current batch, and a gap in test-environment documentation.

Pipeline Stages and Dataset Statistics

Bug Categories in this Dataset

Every bug we work with falls into one of three categories:

- Memory management bugs.
- Graphical/rendering bugs (layout, compositing, GPU crashes)
- Logic/contract violation bugs

Stage 1 — Extract crash bugs

Source: Bugzilla REST API (<https://bugzilla.mozilla.org/home>), scoped to Core + Firefox.

Filter: extract bugs that **resolution = FIXED**, has a populated **crash signature**.

Each bug is classified by its creation date against a 6-month cutoff: recent (created within the past 6 months) or older (created earlier than 6-month). This split matters because Mozilla's Socorro crash-reporting system retains full, complete stack traces for crashes only within a limited creation window; recent bugs can still be linked back to their original Socorro report, while older bugs' Socorro links have since expired, leaving only whatever partial stack trace a developer happened to paste into the bugzilla comment at the time.

Results: 87 recent bugs, 4,664 older bugs kept.

Stage 2 — Extract stack traces

A stack trace serves as our evidence that a real crash actually occurred; it's the execution record showing the browser genuinely faulted at a specific point in the code, as opposed to a bug that was merely reported or assumed. Requiring one for every bug filters out reports that lack this concrete evidence and gives us a starting point for later associating the crash with the code responsible.

For **recent bugs** (bugs within the last 6 months), we fetch the complete stack trace directly from their still-active Socorro crash report (all the stack frames). For **older bugs**, we fall back to the truncated stack trace embedded in the original Bugzilla comment, typically just the top ~10 frames.

Result: 86 bugs with full Socorro traces, 4,665 bugs with partial (~10 frame) traces only.

Stage 3 — Filter by regression (noise-reduction signal only, not used for test validation)

We quarry Bugzilla's **regressed_by** field and keep only bugs where it's populated. This field is Mozilla developers' own attribution that a crash was caused by a specific prior regression bug; we use it purely as a **confidence filter**: a bug with a documented **regressed_by** link is far more likely to be a genuine, well-understood code-caused crash than a bug with no such link (which might be a one-off environment crash, a duplicate, or something without a clear code cause). This gives us a lower-noise starting population, but we do not use the regressor commit itself anywhere downstream.

Result: 835 bugs kept which have a **regressed_by** bug, 3,915 filtered out.

Stage 4 — Find fixing commit and its parent commit

For each bug that passed the regression filter, we identify the **fixing commit** (via Bugzilla history/comments, falling back to Mercurial log search across local **mozilla-central/autoland/reslease** clones — every commit hash is verified by confirming the bug ID actually appears in the commit message) and its **immediate parent commit** in Mercurial history.

We deliberately do not use the regressor commit(s) for test validation. We originally collected **regressed_by**/regressor-commit info because it was central to an earlier version of this pipeline (the static file/method-overlap (between **regressor-commit** and **fixing-commit**) approach). We kept the **regressed_by** field as a Stage 3 noise filter, but a **regressed_by** field frequently points to a **bug**, not a single commit — and that regressor bug itself is often resolved via multiple commits, several of which may be unrelated refactors, follow-ups, or backouts with no direct causal relationships to the specific crash we're studying. Reliably picking one commit among several candidates that actually caused a given crash is exactly the kind of ambiguous attribution problem we're trying to avoid.

Instead, we use the **fixing commit's immediate parent** as the “buggy” revision to test against. This target is unambiguous (exactly one parent per fixing commit in linear history) and guaranteed by construction to be the revision the fix was actually written against; so if the fixing commit's own test can distinguish parent-vs-fix behavior, that's a clean, well-defined validation, with no need to correctly identify a regressor commit among several candidates.

Result: 717 bug matches — each had at least one fixing commit and a resolvable parent commit.

Test Execution Methodology: Patching the Fixing Commit's Test onto the Parent

A structural consequence of using the parent commit as our “buggy” revision: the parent commit predates the fix, so it typically does not yet contain the test file the fixing commit added or modified (that's often the whole point of the fix — introducing a new regression test alongside the code fix). Running the fixing commit's test suite unmodified against the parent checkout would simply fail to find the test at all, not because the underlying test is absent, but because the test file itself doesn't exist yet at that revision.

To test the parent commit meaningfully, we patch in the fixing commit's test file(s) onto the parent's commit's checkout before running: we extract the test file's exact content as it exists at the fixing commit (via `hg cat -r <fixing_commit> <test_file>`) and write it into the parent commit's working directory, without alerting anything else about the parent's actual source code. We then run the test against the parent as patched, and revert the patch afterward before moving to the next commit — the parent commit itself is never modified in Mercurial history, only the local working directory is temporarily alerted for the duration of that one test run.

This is what makes our validation meaningful: if the same test (byte-for-byte identical between both runs) passes against the fixing commit's code and fails against the

parent's code, that's direct evidence the code change between the two revisions is what changed the test's outcome — not a difference in which test ran.

This patching step is also where several of the environment issues below originate – for frameworks with hand-manitained test manifests (mochitest, xpcshell), patching the test file alone is not sufficient, since the manifest that tells the test runner the file exists also needs patching; see “Environment Fixes Applied” below.

Stage 5 — Check for test files on the fixing commit

Pattern-match every file changed in the fixing commit against known Mozilla test path/naming conventions (mochitest, xpcshell, gtest, wpt, crashtest, jst-test).

Result: 345 bugs have at least one test file (proceed to build), 372 bugs java none (excluded; no way to empirically validate the fix without a test).

Stage 6 — Build readiness filter

Before attempting a full build, we discard bugs where every test file is non-runnable in a Linux CI-style environment (Android-only tests, header/manifest-only files, unsupported frameworks like marionette).

Results: 20 bugs filtered out here.

Stage 7 — Artifact build attempt

We try Mozilla's prebuilt-binary (“artifact build”) path first because it's dramatically faster than compiling from source — it downloads Mozilla's own pre-compiled C++ binaries for a given revision and only builds the thin JS/front-end layer locally, cutting build time from potentially hours down to minutes. Given the scale of our dataset (hundreds of commit pairs, each needing both a fixing-commit and parent-commit build), starting with artifact builds lets us validate as many bugs as possible at minimal cost before falling back to the far more expensive full-build path only where necessary.

Results: 27 bugs with prebuilt binaries available; 298 needed a full source build.

Of the 42 with prebuilt binaries: 13 validated (fix passes, parent fails), 14 bugs ran but did not validate (fix fails or parent passes).

Stage 8 — Full build, split by Python era

The remaining 318 bugs had no prebuilt artifact available for one or both of their commits — this is expected, since Mozilla’s artifact server only retains prebuilt binaries for a limited retention window, and **parent commits in particular** are frequently just old enough (one revision earlier than an artifact-covered commit) to fall outside that window even when the fixing commit itself is still covered. For these bugs, there’s no shortcut: we compile Firefox from source directly at each commit, using the exact toolchain versions (clang, cbindgen, Rust) that commit’s era of the build system actually requires.

Of the 318, 20 were excluded by the same readiness filter as Stage 6 (non-runnable tests), leaving **298 buildable bugs**. Firefox’s build toolchain requirements shift over calendar time — most notably, mach’s own Python compatibility; so we split the remaining bugs by which commit’s build system are Python 2-compatible vs. Python3-compatible. We make this classification empirically rather than by commit date. Our build script first attempts a real build with the current Python 3 toolchain, unconditionally, for every commit. Only when that build fails do we inspect the failure output for a fixed set of Python 2-specific signatures — `SyntaxError`, `ImportError: cannot import name`, `references to collection.abc failing to resolve`, Python 2-only constructs (`basestring`, `has_key`, `iteritems`, `itervalues`, `__builtin__`), or the mach virtualenv’s Python interpreter not being found. If any of these appear, we classify the failure as `python2_env` and route the bug into a separate retry queue rather than a generic build failure. If none appear, we classify it as a genuine build failure and leave it in place. This lets us distinguish “this specific commit’s era of mach genuinely predates Python 3 support” from other, unrelated build failures with a documented, reproducible rule rather than a manual judgment call:

- **Python 2 era (2018-2022): 95 bugs — on hold not yet attempted.**

- **Python 3 era (2022-2026): 203 bugs — current active focus.**

Current status of the Python 3 batch

- 58 bugs built successfully.
 - 17 bugs seem to be validated
 - 13 bugs that fail on both fixing and parent commits.
 - 5 bugs that pass on both fixing and parent commits.
 - 19 bugs not validated (fails on fix or pass on parent).
 - 148 bugs blocked on a build failure — this is where we need your input, detailed below.
-

What We're Asking Mozilla Developers

Given the pipeline above, we're currently hitting two categories of problems we'd value your input on.

1. The Build Blocker: cbindgen TOML Parse Failures

After eliminating every environment/toolchain issue we could find (see “Environment Fixes Applied” below), the remaining 136 bugs all fail at the same configure/build step, reproducing identically on repeat attempts, with variants of:

```
0:01.83 thread 'main' (891375) panicked at
/home/cc/.cargo/registry/src/index.crates.io-1949cf8c6b5b557f/cbindgen-0.29.2/src/bindgen/config.rs:1130:34:
0:01.83 called `Result::unwrap()` on an `Err` value: "Couldn't parse config file:
TOML parse error at line 343, column 1\n  |\n343 | \"Keyframe\" =
\"Keyframe\"\n  | ^^^^^^^^^^^^^\nduplicate key\n."
```

```
0:01.83 note: run with `RUST_BACKTRACE=1` environment variable to display
a backtrace
0:01.83 ./rust_minidump_writer_linux_ffi_generated.h.stub
0:01.84 gmake[3]: *** [backend.mk:596:
layout/style/.deps/ServoStyleConsts.h.stub] Error 101
0:01.84 gmake[3]: *** Waiting for unfinished jobs....
0:02.26 gmake[2]: ***
[/data/FaultLocalizationIndustry/mozilla-central/config/recurse.mk:34: export]
Error 2
0:02.26 gmake[1]: ***
[/data/FaultLocalizationIndustry/mozilla-central/config/rules.mk:361: default] Error
2
0:02.26 gmake: *** [client.mk:60: build] Error 2
0:02.26 W 0 compiler warnings present.
```

We traced this to the source directly: [servo/ports/geckolib/cbindgen.toml](#) contains the literal key **"Keyframe" = "Keyframe"** twice, which is invalid under the TOML specification itself. We confirmed that across 136 bugs there are 234 occurrences of this exact duplicate, spanning December 2021 through August 2024. Checking the current tip confirms that the file now contains only a single occurrence — the duplicate has since been resolved, though we haven't identified the specific commit that fixed it.

As a quick diagnostic, we tested removing the duplicate locally (a throwaway edit, reverted immediately, never persisted) on one affected commit. The build proceeded further but hit a second, separate cbindgen error, so the duplicate key is not necessarily the sole blocker for every affected commit — there may be a second, related incompatibility between our cbindgen version and Rust source patterns from this era.

Questions for you:

- Is this known historical issue with [servo/ports/geckolib/cbindgen.toml](#) ? Do you know roughly when/why the duplicate key was introduced, and when it was resolved?
- Given the file is genuinely invalid TOML, do you know whether the cbindgen version in CI use during 2021-2024 simply didn't enforce strict duplicate-key rejection?
- Is there a recommended cbindgen version (or version-per-era mapping) we should use for commits in this window, rather than the current pin?
- Would removing the duplicate key locally (as a build-only patch, never persisted to the actual commit) be a safe way to unblock affected commits for our purposes, or could that mask a deeper compatibility issue that we should understand first?

2. Is There a Way to Avoid Manually Discovering Dependencies Per Test Framework?

We're following Mozilla's own published documentation for running each test suite – especially:

- <https://firefox-source-docs.mozilla.org/testing/mochitest-plain/index.html>
- <https://firefox-source-docs.mozilla.org/testing/browser-chrome/index.html>
- <https://firefox-source-docs.mozilla.org/web-platform/index.html>
- <https://firefox-source-docs.mozilla.org/js/test.html>
- <https://firefox-source-docs.mozilla.org/gtest/index.html>

These docs are clear and accurate about **command syntax** – flags, invocation patterns, how each framework discovers tests. What they don't cover is **environment provisioning**: what system packages and libraries need to be installed before a given test framework will even launch, especially on a bare linux instance with no prior Firefox development history.

Concretely, this is what we ran into: running `./mach bootstrap` gets you the core build toolchain (clang, rustc, cbindgen, node, nasm), but it does not cover runtime dependencies needed to actually execute tests. We only discovered the

following were missing by running each framework, reading the resulting error, installing the missing piece, and repeating – sometimes many times in a row for a single framework:

- **General test runtime:** `gsettings-desktop-schemas`, `xvfb` + a running `Xvfb :99 display`, `libgtk-3-0`, `fontconfig`
- **WPT specifically:** `aioquic`, `pyOpenSSL` (via `./mach python -m pip install`, not system pip — this isn't obvious from the docs)
- **gtest specifically:** an entire chain of system libraries only surfaced when a full configure ran for the first time on a fresh instance — `pkg-config`, `libasound2-dev`, `libpulse-dev`, `libclang-17-dev` (plus manual unversioned symlinks: `clang-17` → `clang`, `llvm-objdump-17` → `llvm-objdump`, since Mozilla's build tooling looks for unversioned names even when only versioned LLVM packages exist), the full GTK/X11 dev-header set, and `libstdc++-12-dev` (needed because clang found GCC 12's compiler but only GCC 11's actual C++ standard library headers were present — fatal error: 'type_traits' file not found)

None of this is documented in the per-framework testing docs above, and it's not obvious in advance which framework will need which subset. We found out one missing library at a time, per-framework, across several fresh-instance rebuilds.

Specifically, we'd like to know:

- Is there a Mozilla-maintained Docker image, worker image, or provisioning script (something used internally to automatically build and test every commit) that already has every dependency for all six frameworks pre-installed, that we could either use directly or read as a reference for what a complete dependency list looks like?
- Alternatively, is there a documented 'run any test suite' entry point; something like `mach bootstrap --testing` or similar that installs test-runtime dependencies the same `mach bootstrap` installs build-time ones, that we're simply missing?

- If no such single source exists, would it be useful to Mozilla for us to contribute to the consolidated dependency list we've built.

We are not looking for remote CI execution access to run tests remotely; we specifically need local, arbitrary historical commit build+test capability (parent commits specially are often too old for `mach artifact install` to find a prebuilt binary for), so we expect to keep building locally regardless. The ask here is purely about closing the dependency discovery gap, not about finding a remote execution alternative.

Environment Fixes Applied (for context – what we've already ruled out)

We rebuilt our environment from scratch this session and systematically resolved, in order: missing Mercurial, missing `pip3`, a stale/wrong-era `cbindgen` (bootstrap installs whatever tip currently requires — 0.29.x — rather than the version historical commits need), a similarly wrong-era `clang` (bootstrap gives clang 20; we pinned clang 17 via LLVM's official apt repo instead), missing `m4`, and the full gtest-specific system library chain (`llvm-objdump symlink`, `pkg-config`, `libasound2-dev`, `libpulse-dev`, `libclang-17-dev` + symlinks, GTK/X11 dev headers, `libstdc++-12-dev`). We also maintain a running list of framework-specific recurring fixes:

- **WPT:** a stale vendored `typing_extensions.py` under `testing/web-platform/tests/tools/third_party/` resets on every `hg update --clean` and must be reapplied from the wpt venv's current copy before every WPT-family run.
- **mochitest/mochitest-browser:** `CLOBBER` staleness after every saved-build restore (`TestManifestBackend` is out of date); for parent-commit reruns, both the

test file *and* its manifest entry (`mochitest.toml/.ini`, `xpcshell.toml/.ini`, etc.) must be patched in from the fixing commit, since these manifests are hand-maintained, not auto-regenerated.

- **jit-test:** `mach_command` strings need the `js/src/jit-test/tests/` prefix stripped before invocation, or `./mach jit-test` silently matches zero tests (`returncode 0`, empty output) and our generic result-parsing defaults to opposite pass/fail for fixing vs. parent.
- **gtest:** targeted `./mach gtest <filter>` runs against a freshly-restored saved build reliably fail with `Could not build xul-gtest`, since that library is only ever linked by a full `./mach build`, never a targeted/incremental one — we now unconditionally run a full build before any gtest filter, even on a cache hit.
- **WPT crashtests** (files under `testing/web-platform/tests/.../crashtests/`) are a distinct WPT test type and cannot run via `./mach crashtest` (the in-tree Gecko harness) — they need `./mach wpt --test-type=crashtest` specifically.

We're confident these are correctly resolved (confirmed via successful builds/validations across multiple frameworks, including bugs that build and ran but did not validate); the TOML duplicate-key issue is the one category we can't work around ourselves. That said, everything above was discovered through manual trial and error, one missing dependency at a time — see our questions above about whether a pre-provisioned image or documented dependency list already exists that would let us (and future contributors) skip this discovery process entirely.

Pipeline Summary Diagram





